

UNIT IV – MULTIMODAL GENERATIVE AI APPLICATIONS

QUESTION 6 – TEXT-TO-SPEECH APPLICATION

Engineering Text to Natural-Sounding Speech

Professional Implementation Report

Question	Develop a Text-to-Speech application that converts engineering-related text into natural-sounding speech using a pre-trained AI model.
Folder Name	Q6 Text To Speech
Python File	text_to_speech.py
Interface	Streamlit Web Application
AI Service	Hugging Face Inference API with a pre-trained Text-to-Speech model

1. AIM

To develop a Text-to-Speech application that accepts engineering-related text and converts it into natural-sounding speech using a pre-trained AI speech-generation model.

2. OBJECTIVES

- Accept engineering-related text from the user.
- Convert the text into speech using a pre-trained AI model.
- Provide an interactive Streamlit interface.
- Play the generated speech directly in the browser.
- Allow the generated audio to be downloaded for later use.
- Demonstrate a practical Generative AI application for engineering education.

3. SOFTWARE AND HARDWARE REQUIREMENTS

- Python 3.x
- Streamlit
- Hugging Face Hub / Inference API
- Pre-trained Text-to-Speech model
- Internet connection for API inference
- Computer or laptop with audio playback support

4. FOLDER AND FILE STRUCTURE

UNIT_IV_Multimodal_Generative_AI
└─ Q6_Text_To_Speech
 └─ text_to_speech.py

5. TECHNOLOGIES USED

Technology	Purpose	Role in Application
Python	Programming	Implements the application
Streamlit	Web interface	Accepts engineering text and displays audio
Hugging Face Inference API	AI inference	Connects application to pre-trained speech model
InferenceClient	API client	Sends text and receives generated audio
Pre-trained TTS model	Speech generation	Converts text into natural-sounding speech

6. SYSTEM WORKFLOW

Engineering Text → Streamlit Interface → Hugging Face Inference API → Pre-trained TTS Model → Generated Audio → Audio Player / Download

Text-to-Speech System Workflow

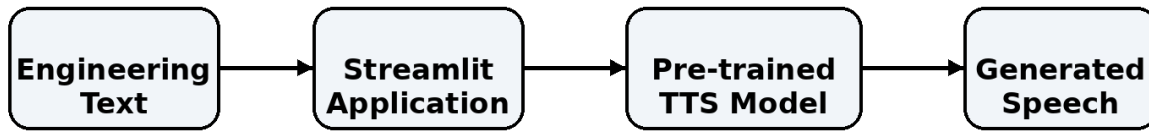


Figure 4. Text-to-Speech system workflow.

The user enters engineering-related text in the Streamlit application. The text is sent to a pre-trained text-to-speech model through the Hugging Face inference service. The returned audio is played in the browser and can also be downloaded.

7. STEP-BY-STEP IMPLEMENTATION

Step 1: Create the Q6_Text_To_Speech folder inside UNIT_IV_Multimodal_Generative_AI.

Step 2: Create the Python file text_to_speech.py.

Step 3: Install Streamlit and huggingface_hub.

Step 4: Create a Hugging Face access token and keep it private.

Step 5: Set the HF_TOKEN environment variable.

Step 6: Initialize the Hugging Face InferenceClient.

Step 7: Create a text area for entering engineering-related text.

Step 8: Validate that the text input is not empty.

Step 9: Send the text to the pre-trained Text-to-Speech model.

Step 10: Receive the generated audio bytes.

Step 11: Display the generated speech using Streamlit's audio player.

Step 12: Provide a download button for the generated speech file.

Step 13: Test the application with multiple engineering statements.

8. COMPLETE IMPLEMENTATION CODE

```
import streamlit as st
from huggingface_hub import InferenceClient
import os

st.set_page_config(
    page_title="Engineering Text-to-Speech",
    page_icon="🔊",
    layout="centered"
)
```

```

st.title("🔊 Engineering Text-to-Speech Application")

st.write(
    "Enter engineering-related text and convert it into "
    "natural-sounding speech using a pre-trained AI model."
)

HF_TOKEN = os.getenv("HF_TOKEN")

if not HF_TOKEN:
    st.error(
        "HF_TOKEN is not configured. "
        "Please configure your Hugging Face token."
    )
    st.stop()

client = InferenceClient(api_key=HF_TOKEN)

text = st.text_area(
    "Engineering Text:",
    value=(
        "Artificial intelligence is widely used in engineering "
        "to automate complex tasks, analyze data, and improve "
        "decision making."
    ),
    height=180
)

if st.button("🔊 Convert Text to Speech"):

    if not text.strip():
        st.warning("Please enter engineering-related text.")

    else:
        try:

            with st.spinner(
                "Generating natural-sounding speech..."
            ):

                audio_bytes = client.text_to_speech(
                    text=text,
                    model="hexgrad/Kokoro-82M"
                )

                st.success(
                    "Speech generated successfully!"
                )

            st.subheader("🔊 Generated Speech")

```

```

        st.audio(
            audio_bytes,
            format="audio/flac"
        )

        st.download_button(
            label="📄 Download Speech",
            data=audio_bytes,
            file_name="engineering_speech.flac",
            mime="audio/flac"
        )

    except Exception as e:

        st.error(
            "Text-to-speech generation failed."
        )

        st.write(str(e))

st.divider()

st.subheader("💡 Example Engineering Text")

st.write(
    "• Operating systems manage computer hardware and software resources."
)

st.write(
    "• Machine learning enables engineers to analyze data and make intelligent predictions."
)

st.write(
    "• Artificial intelligence can improve engineering design and automation."
)

st.divider()

st.caption(
    "Powered by Hugging Face Inference API and a "
    "pre-trained text-to-speech model."
)

```

9. EXECUTION COMMAND

```
streamlit run ".\Q6_Text_To_Speech\text_to_speech.py"
```

10. TEST INPUTS

- Artificial intelligence is widely used in engineering to automate complex tasks, analyze data, and improve decision making.
- An operating system manages processes, memory, files, and input output devices.
- Machine learning enables engineers to analyze data and make intelligent predictions.

11. IMPLEMENTATION RESULTS AND SCREENSHOTS

The following screenshots show successful execution of the application. Each test demonstrates that engineering-related text was converted into playable speech and that the generated audio could be downloaded.

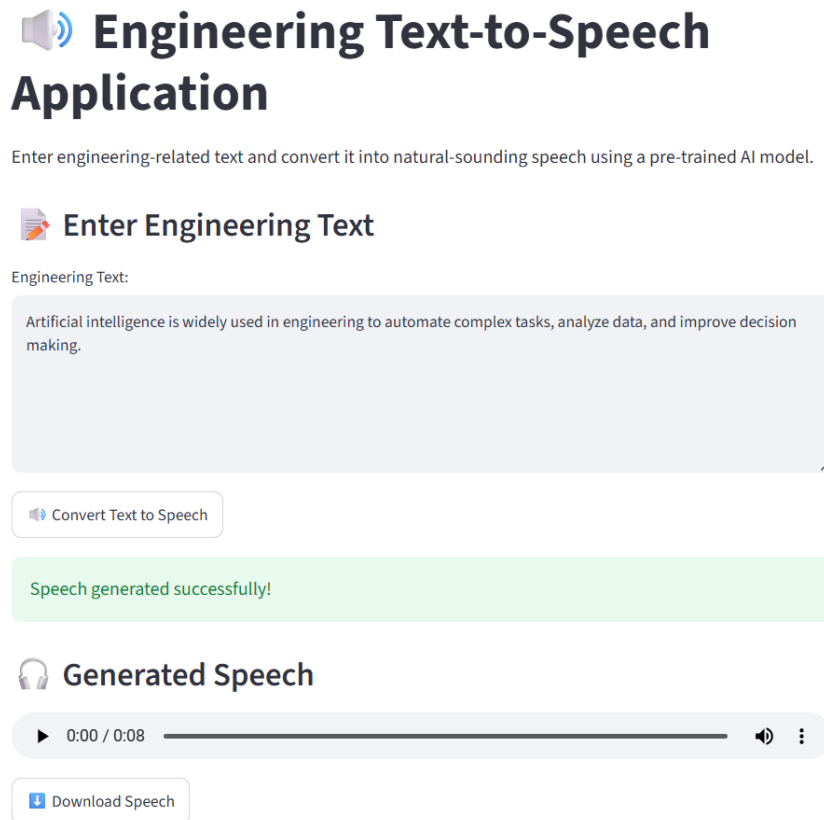


Figure 1. Text-to-Speech application successfully converting an AI engineering statement into speech.

Engineering Text-to-Speech Application

Enter engineering-related text and convert it into natural-sounding speech using a pre-trained AI model.

Enter Engineering Text

Engineering Text:

An operating system manages processes, memory, files, and input output devices.

 Convert Text to Speech

Speech generated successfully!

Generated Speech

▶ 0:00 / 0:06   

 Download Speech

Figure 2. Text-to-Speech application converting an operating-system statement into generated speech.

Engineering Text-to-Speech Application

Enter engineering-related text and convert it into natural-sounding speech using a pre-trained AI model.

Enter Engineering Text

Engineering Text:

Machine learning enables engineers to analyze data and make intelligent predictions.

 Convert Text to Speech

Speech generated successfully!

Generated Speech

▶ 0:00 / 0:05



 Download Speech

Figure 3. Text-to-Speech application converting a machine-learning statement into generated speech.

12. TEST RESULTS

Test Case	Engineering Text	Observed Output
1	Artificial intelligence is widely used in engineering to automate complex tasks, analyze data, and improve decision making.	Speech generated successfully; audio player displayed.
2	An operating system manages processes, memory, files, and input output devices.	Speech generated successfully; audio player displayed.
3	Machine learning enables engineers to analyze data and make intelligent predictions.	Speech generated successfully; audio player displayed.

13. RESULT

The Engineering Text-to-Speech application was successfully developed and tested. The application accepts engineering-related text, sends it to a pre-trained speech-generation model through the

Hugging Face inference service, and returns playable natural-sounding speech. The screenshots demonstrate successful generation for artificial intelligence, operating systems and machine learning topics.

14. ADVANTAGES

- Converts engineering text into speech automatically.
- Uses a pre-trained AI model without requiring model training.
- Provides immediate browser-based audio playback.
- Allows users to download generated speech.
- Useful for accessibility, presentations and audio-based learning.

15. LIMITATIONS

- Internet access is required for API-based speech generation.
- Audio quality depends on the selected model and inference service.
- API limits or service availability can affect generation time.
- The current application uses a fixed language/model configuration.

16. FUTURE ENHANCEMENTS

- Add multiple Indian-language voices.
- Provide voice and language selection options.
- Allow users to control speaking speed and style.
- Generate downloadable MP3/WAV audio formats.
- Connect the system with the Speech-to-Text chatbot for two-way voice interaction.

17. CONCLUSION

The developed Text-to-Speech application demonstrates how multimodal Generative AI can convert engineering-related written content into natural-sounding audio. The successful test cases confirm that the application can generate and play speech through a simple Streamlit interface. The system can be further enhanced with multilingual voices, speaking controls and integration with other AI applications.

18. VIVA / KEY POINTS

- Text-to-Speech converts written text into spoken audio.
- A pre-trained TTS model is used instead of training a speech model from scratch.
- Hugging Face Inference API provides model access.
- Streamlit is used for the interactive web interface and audio playback.
- The generated audio can be downloaded for later use.